Though token introspection solves the problem of how the API can determine if an access token is valid and the scope associated with that token, it has a downside: the API must make a call to the AS every time it needs to validate a token. An alternative is to use a self-contained token format such as JWTs that were covered in chapter 6. This allows the API to validate the access token locally without needing to make an HTTPS call to the AS. While there is not yet a standard for JWT-based OAuth2 access tokens (although one is being developed, see https://tools.ietf.org/html/draft-ietf-oauth-access-token-jwt-00), it's common for an AS to support this as an option.

To validate a JWT-based access token, the API needs to first authenticate the JWT using a cryptographic key. In chapter 6 you used symmetric HMAC or authenticated encryption algorithms in which the same key is used to both create and verify messages. This means that any party that can verify a JWT is also able to create one that will be trusted by all other parties. Although this is suitable when the API and AS exist within the same trust boundary, it becomes a security risk when the APIs are in different trust boundaries. For example, if the AS is in a different datacenter to the API, the key must now be shared between those two datacenters. If there are many APIs that need access to the shared key, then the security risk increases even further because an attacker that compromises any API can then create access tokens that will be accepted by all of them.

To avoid these problems, the AS can switch to public key cryptography using digital signatures, as shown in figure 7.8. Rather than having a single shared key, the AS instead